Sessioni, resume, fork e contesto dei subagent

> Appunti per la certificazione CCA-F (Domini 1 e 5). Spiegano come **riprendere**, **biforcare** e **isolare** il lavoro di Claude, e cosa **vede davvero** un subagent.
>
> ⚠️ Flag e nomi delle opzioni possono cambiare tra versioni. Per i dettagli aggiornati fa fede la documentazione ufficiale: **docs.claude.com** → **Claude Code** e **Agent SDK** → **Sessions**.

Indice

1. Cos'è una sessione
2. Le quattro strade: continuare, riprendere, biforcare, ripartire
3. Comandi in Claude Code
4. Sessioni nell'Agent SDK
5. Riprendere quando qualcosa è cambiato
6. Il contesto dei subagent
7. Isolare il lavoro esplorativo: `context: fork`
8. Gestire il contesto lungo: compact e clear
9. Trappole e domande da esame

---

1. Cos'è una sessione

Una **sessione** è la cronologia di una conversazione con Claude: messaggi, chiamate ai tool e loro risultati. Viene salvata con un **session ID**, quindi può essere ripresa in seguito.

In Claude Code le trascrizioni stanno in `~/.claude/projects/<progetto>/` e vengono conservate per `cleanupPeriodDays` giorni.

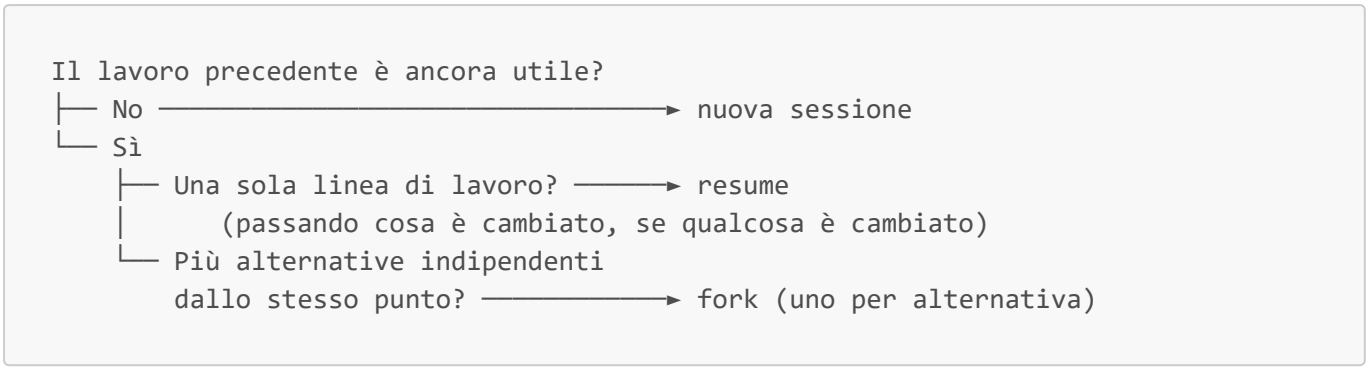
> 💡 Una sessione conserva **ciò che Claude ha visto**, non lo stato attuale dei file. Se i file cambiano dopo, la sessione contiene informazioni potenzialmente **superate**.

---

2. Le quattro strade: continuare, riprendere, biforcare, ripartire

Strategia	Cosa succede	Quando usarla
Continuare	Riapre l'ultima sessione	Riprendere subito il lavoro interrotto
Riprendere (resume)	Riapre una sessione specifica tramite ID	Continuare un lavoro di giorni fa, con tutta la sua storia
Biforcare (fork)	Crea una **nuova** sessione che parte dalla storia di quella originale, che resta intatta	Esplorare alternative dallo stesso punto di partenza senza contaminazione
Ripartire da zero	Nuova sessione vuota	Il lavoro precedente è irrilevante o inaffidabile, o il contesto è "sporco"

Come scegliere



Esempio d'esame: tre piani alternativi

Dopo un'analisi approfondita, gli architetti vogliono esplorare **event sourcing**, **irrigidimento dello schema** e **cache/read-model**, partendo tutti dalla stessa analisi ma senza che le ipotesi di un piano influenzino gli altri.

Opzione	Valutazione
Tutti e tre nella stessa sessione, con separatori	❌ Le ipotesi si contaminano
Resume per uno, sessioni nuove con note per gli altri	❌ Un ramo ha il contesto completo, gli altri no
Tre sessioni nuove con un breve riassunto	❌ Si perde il dettaglio dell'analisi
Tre fork dallo stato post-analisi	✅ Stessa base completa, evoluzione indipendente

---

3. Comandi in Claude Code

Comando	Effetto
`claude --continue` (`-c`)	Continua la sessione più recente nella cartella corrente
`claude --resume` (`-r`)	Mostra l'elenco delle sessioni da riprendere
`claude --resume <session-id>`	Riprende una sessione specifica
`claude --resume <id> --fork-session`	Riprende creando un **fork**: nuovo ID, l'originale resta invariata
`/resume`	Riprende una sessione dall'interno di Claude Code
`/clear`	Svuota il contesto e inizia da capo
`/compact [istruzioni]`	Riassume la conversazione per liberare contesto

In modalità non interattiva (`claude -p`) le stesse opzioni permettono di concatenare esecuzioni in CI mantenendo il contesto tra un passo e l'altro.

---

4. Sessioni nell'Agent SDK

Il session ID si ottiene dal primo messaggio di sistema (`init`) restituito dalla query e si salva per riprenderlo in seguito.

```python
from claude_agent_sdk import query, ClaudeAgentOptions

# Riprendere una sessione salvata
async for msg in query(
    prompt="Continua a generare gli handler. Sono cambiati:
src/Inventory/OrderProjector.cs e schemas/warehouse_events.json; ricontrollali
prima.",
    options=ClaudeAgentOptions(resume=session_id),
):
    ...

# Biforcare: nuova sessione con la stessa storia, l'originale non viene modificata
async for msg in query(
    prompt="Progetta la soluzione basata su event sourcing.",
    options=ClaudeAgentOptions(resume=session_id, fork_session=True),
):
    ...
```

Opzione	Effetto
`resume=<id>`	Continua la sessione indicata, aggiungendo alla sua storia
`resume=<id>` + `fork_session=True`	Parte dalla storia della sessione indicata ma crea una **nuova** sessione

> ⚠️ Il fork **copia la storia**, non aggiunge requisiti nuovi: le nuove istruzioni vanno comunque scritte nel prompt.

---

5. Riprendere quando qualcosa è cambiato

Scenario tipico: due giorni dopo, il sistema di build dice che sono cambiati solo due file.

Opzione	Valutazione
Resume e rilettura di **tutti** i file analizzati	❌ Spreco di tempo e contesto
Nuova sessione da zero	❌ Butta via lavoro valido
Resume senza menzionare i cambiamenti	❌ L'agente si basa su informazioni superate
Resume passando l'elenco dei file cambiati	✅ Ricontrolla solo ciò che serve e prosegue

> 💡 Regola generale: **riprendi e informa**. La sessione ripresa non sa cosa è successo nel frattempo; sei tu a dirglielo.

---

6. Il contesto dei subagent

È il punto più frainteso, e all'esame compare spesso.

Un **subagent** (lanciato dal coordinatore con il tool **Task**) parte con un **contesto nuovo e separato**:

Il subagent **vede**	Il subagent **non vede**
Il proprio system prompt (definizione dell'agente)	La cronologia della conversazione del coordinatore
Il **prompt** che il coordinatore gli passa nella chiamata	Gli altri subagent e i loro risultati, se non glieli passi
I tool che gli sono stati concessi	Le decisioni prese dall'utente in altri turni

E al ritorno:

- Il subagent restituisce al coordinatore **solo il risultato finale**, non tutta la sua esplorazione.
- È proprio questo che **protegge il contesto** del coordinatore.

Conseguenza pratica

> Se i requisiti cambiano, il coordinatore deve **ripeterli esplicitamente in ogni nuova invocazione** dei subagent.

Esempio d'esame: dopo `--resume`, il product lead restringe il perimetro (solo fintech europee, stipendi in EUR, dati non più vecchi di 18 mesi, fonti ordinate per verificabilità).

Opzione	Valutazione
Abbandonare la sessione e ripartire	❌ Butta via il lavoro fatto
Invocare i subagent "normalmente"	❌ Non leggono i messaggi del coordinatore
Fare un fork e aspettarsi che i requisiti si propaghino	❌ Il fork non inietta requisiti nei subagent
Resume, riassumere i risultati, includere perimetro, regole e criteri in ogni invocazione	✅

Cosa mettere nel prompt del subagent

- Obiettivo preciso e criterio di completamento.
- Vincoli e requisiti aggiornati (perimetro, formati, fonti ammesse).
- Il contesto rilevante già raccolto (un riassunto, non l'intera storia).
- Il **formato di output** atteso, possibilmente strutturato.

---

7. Isolare il lavoro esplorativo: `context: fork`

Nel frontmatter di una **skill**, `context: fork` la esegue in un **subagent isolato**:

```
---
name: dependency-review
description: Esamina le modifiche al lockfile e individua aggiornamenti rischiosi
```

```
context: fork
---
```

Effetto	Sì / No
Le letture esplorative restano fuori dalla sessione principale	✅
La sessione principale riceve solo il risultato finale	✅
Crea un fork Git del repository	❌
Gira in parallelo mentre continui a lavorare	❌
Scrive tutte le note nella conversazione principale	❌ (è l'opposto)

> 💡 Da non confondere: `context: fork` (skill isolata in un subagent) ≠ `--fork-session` / `fork_session` (copia di una sessione per esplorare alternative).

---

8. Gestire il contesto lungo: compact e clear

Strumento	Cosa fa	Quando
`/compact`	Riassume la storia mantenendo i punti essenziali	Sessione lunga ma ancora sullo stesso compito
Compattazione automatica	Avviene vicino al limite del contesto	Automatica; l'hook `PreCompact` può salvare informazioni prima
`/clear`	Svuota completamente il contesto	Cambio di compito; contesto pieno di tentativi falliti
Subagent	Delegano l'esplorazione e restituiscono solo il risultato	Ricerche, review, letture voluminose
File di appunti / memoria	Persistono fatti importanti fuori dalla conversazione	Lavori lunghi su più sessioni

> ⚠️ Un riassunto perde dettagli. Fatti critici (numeri, ID, decisioni, vincoli) vanno conservati in modo esplicito, ad esempio in un file o in un blocco "fatti chiave" riportato a ogni passo.

---

9. Trappole e domande da esame

Trappole

- ❌ Pensare che un subagent "sappia" cosa ha detto l'utente al coordinatore.
- ❌ Pensare che un fork propaghi automaticamente nuovi requisiti.
- ❌ Riprendere una sessione senza dire cosa è cambiato nel codice.
- ❌ Esplorare alternative nella stessa sessione (contaminazione).
- ❌ Confondere `context: fork` con un fork Git o con l'esecuzione in parallelo.